

Karpenter 太黑盒？事件驱动给它“开窗”！让

自动扩缩容变得可观测与可审计

——Karpenter 很聪明，但我们需要理解它的决策

张 海 涛 & 罗 银 萍

Software Engineer

2025/11/15



张海涛

CloudPilot AI Inc. Software Engineer

Haitao Zhang (GitHub@helen-frank) is a major contributor and reviewer of karpenter-provider-alibabacloud, and a member of kubernetes-sigs and karmada. Currently focusing on the cloud node autoscale field.

CONTENT

目录

01 什么是Karpenter

02 为何说Karpenter是黑盒

03 Karpenter Event

04 Q&A

什么是 Karpenter

简介

Karpenter 是一个强大的Kubernetes节点自动缩放器，专为灵活性、性能和简单性而构建。它自动配置计算资源，以响应无法安排的pod，与传统的集群自动缩放器相比，实现更快的扩展和更好的利用率。

Karpenter 通过以下方式提高在 Kubernetes 集群上运行工作负载的效率和降低成本：

- 监视Kubernetes 调度器标记为不可调度的 Pod。
- 评估Pod 请求的调度约束（资源请求、节点选择器、亲和性、容忍度和拓扑分布约束）
- 配置满足 Pod 要求的节点
- 当节点不再需要时，将其移除。

什么是 Karpenter

已公开支持的Karpenter Cloud Providers

- **AWS**
- **Azure**
- **AlibabaCloud**
- **Bizfly Cloud**
- **Cluster API**
- **GCP**
- **IBM Cloud**
- **Proxmox**
- **Oracle Cloud Infrastructure (OCI)**

什么是 Karpenter

Spot insight – spot.cloudpilot.ai

ecs.g5.large

ecs.g5.large

Instance Spec

Compute	Value
Arch	amd64
CPU	2
Pyphysical Processor	Intel Xeon (Skylake) Platinum 8163 / Intel Xeon (Cascade Lake) Platinum 8269CY
Clock Speed	2.7 GHz
Memory	8 GiB

Tired of Spot Instance Interruptions?


Book a demo to see how CloudPilot AI helps you minimize interruptions to nearly 0 and maximize your cloud savings. Schedule your demo now!

Instance Family

Name	Arch	vCPUs	Memory (GiB)
ecs.g5.large	amd64	2	8
ecs.g5.xlarge	amd64	4	16
ecs.g5.2xlarge	amd64	8	32

Spot Instance Interruption Map(scroll to zoom)



Extremely Low

Low

Medium

High

Very High

Instance Pricing

Hangzhou, China

Per Hour

On Demand ¥0.89 cn-hangzhou	Spot Min ¥0.098 cn-hangzhou-g	Spot Max ¥0.143 cn-hangzhou-b
	-89%	-83.9%

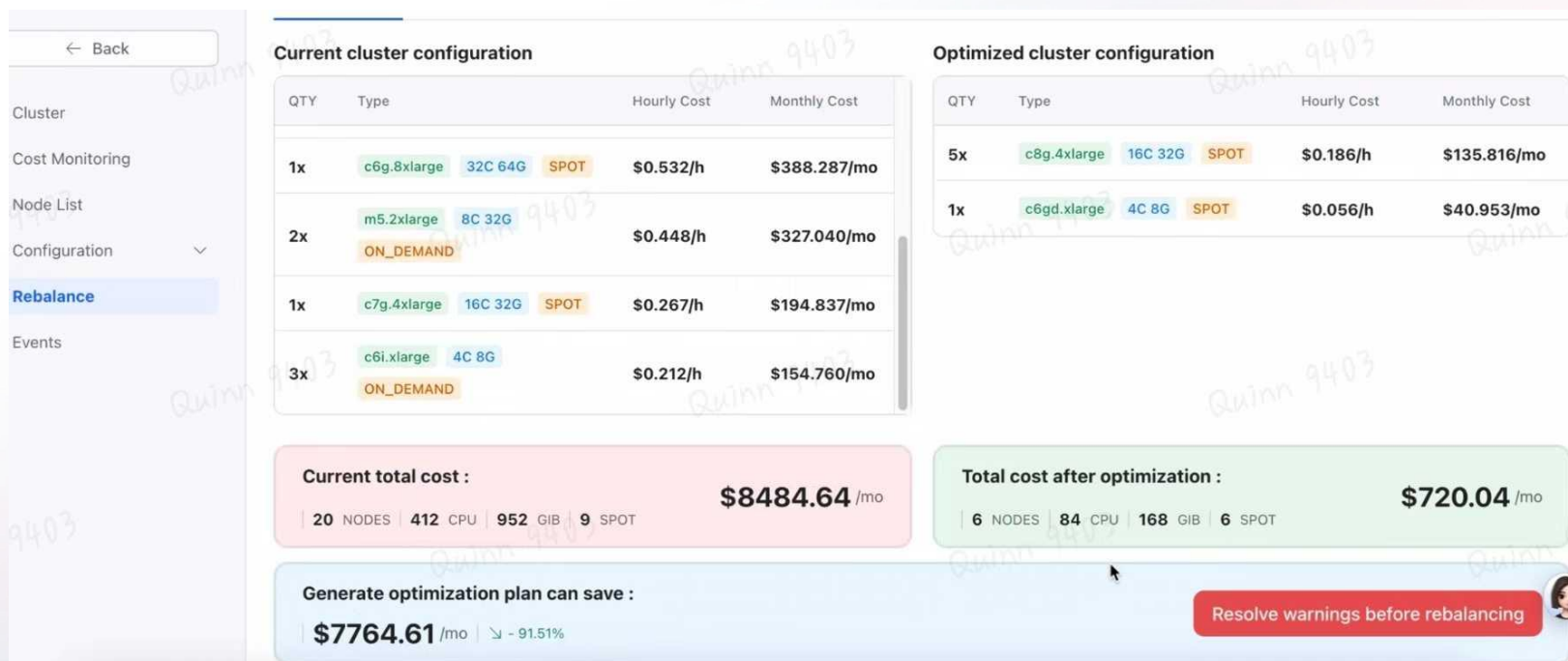
Spot Price Chart

1W 1M 3M



什么是 Karpenter

真实用户节省效果PO图



为何说 Karpenter 是“黑盒”？

自动化决策链路复杂，行为不透明

Karpenter 作为开源的 Kubernetes 节点自动扩缩容工具，其核心优势在于高度自动化，但也正因这一特性，常被用户称为“黑盒”

开源代码虽可见，但运行时决策逻辑不透明。这种矛盾源于其自动化流程的复杂性：系统能自主触发节点创建/删除，却缺乏对决策过程的清晰解释，导致运维人员难以追溯原因

为何说 Karpenter 是“黑盒”？

核心问题: 决策透明度缺失

1. 节点生命周期操作无上下文

- 节点创建/删除的触发原因（如资源需求、成本优化）未被记录，仅能观察结果，无法关联到具体 Pod 或集群事件。

2. 调度失败难以诊断

- Pod 卡在 Pending 状态时，仅显示最终状态，无法还原决策链（例如：为何调度器拒绝分配？是资源不足、标签不匹配，还是节点未及时就绪？）。

3. 云 API 交互不透明

- 云厂商 API 调用失败或延迟时，缺失关键元数据：
 - 具体时间戳？
 - 请求/响应内容（如 AWS EC2 API 的错误代码）？
- 仅依赖日志碎片化信息，定位问题效率低下。

4. 缺乏可视化交互界面

- 所有操作依赖原始日志分析（如 kubectl logs），无图形化视图展示节点决策流、资源依赖或 API 时序，大幅增加理解成本。

直接后果: 排查速度慢, 误判风险高, 运维压力激增

为何说 Karpenter 是“黑盒”？

真实大规模使用karpenter终端用户的声音

“主要是想搞清楚Karpenter做了，然后为什么这么做。删除和增加机器是否是预期的。有的时候pod一直pending在那边，一看Karpenter的日志显示说没有满足的机器。但是里面因为涉及到一些容量计算和一些schedule的配置，需要花点精力才能搞清楚是什么原因导致的。如果出现异常的时候能清晰简单的知道是什么原因导致的，那就完美了”

Karpenter Event

设计目标

- 不改变核心决策逻辑：零侵入主路径
- 结构化、可解析的事件数据
- 高可靠性：异步、重试、容错
- 支持链路还原：时间戳、耗时、错误与请求/响应快照
- 数据清洗：移除冗余/敏感字段，减小负载并保护隐私

Karpenter Event

方案概览

新增组件：EventClient（内嵌于 Provider）

上报：HTTP POST 到 远端（异步队列，默认容量 10000）

关键触发点：

- NodeClaim 创建（apply） / 删除（delete）
- 控制器Command（disruption/provisioner）执行结果
- 上报内容：结构化 JSON（包含 Node、Pod、Provider 请求/响应、时间与错误）

Karpenter Event

事件类别

NodeClaimEvent (Node事件)

- 核心数据: NodeClaim、NodeName、Type (apply/delete) 、Provider、Data (NodeClaim、NodeClass、**ProviderInfo**(可还原云 API 调用))

ControllerEvent (控制器事件)

- 核心数据: ID、Type (disruption/provisioner) 、OperationType (create/delete/replace/no-op) 、NodeClaimNames、DeletedNodeClaimNames、SchedulableFailedPods、Data (DisruptionEvent/ProvisionerEvent)

Karpenter Event

自动重试和可靠性保障

重试机制： `wait.PollUntilContextTimeout`，聚合错误后暴露

非阻塞： 事件发送失败不会影响主流程（主流程的 `defer`/回调只是推入队列）

错误记录： 在 `client.do` 中记录 HTTP 返回码与 `response body`，用于诊断

总之尽最大努力上报，但不牺牲集群稳定性

Karpenter Event

具体设计

事件客户端实现要点:

- 异步队列: nodeClaimEventCh / controllerEventCh
- 发送策略: 循环重试 (最多 5 分钟等待, 间隔 1s)
- 请求超时: HTTP Client timeout (示例 5s)
- Stop/Start: operator 启动时初始化并启动 client, stop 时关闭

数据清洗:

- 使用 CleanPod / CleanNode 等方法移除冗余/敏感字段, 减小负载并保护隐私

Provider 层钩子:

- createHandler / deleteHandler / provisioningHandler / distributionHandler 在关键路径收集上下文并异步上报

流程: 事件收集 -> 清洗 -> 推入队列 -> 异步发送 -> Agent

Karpenter Event

真实案例1

场景：用户大量 Node 创建失败，影响业务

传统：看 Karpenter 日志，信息巨量，排查时间较长

Karpenter event：可以通过UI或者存储查看最近存在错误的NodeClaimEvent ProviderReq/Resp，确认云 API 错误码或限额问题

收益：快速定位到云端限额、权限或参数错误，缩短 MTTR

```
createFleetInput := cfiBuilder.Build()

createFleetOutput, err := p.ec2Batcher.CreateFleet(ctx, createFleetInput)
p.subnetProvider.UpdateInFlightIPs(createFleetInput, createFleetOutput, instanceTypes,
lo.Values(zonalSubnets), capacityType)
if err != nil {
    reason, message := awserrors.ToReasonMessage(err)
    if awserrors.IsLaunchTemplateNotFound(err) {
        for _, lt := range launchTemplateConfigs {
            p.launchTemplateProvider.InvalidateCache(ctx, aws.ToString(lt.
LaunchTemplateSpecification.LaunchTemplateName), aws.ToString(lt.
LaunchTemplateSpecification.LaunchTemplateId))
        }
        return ec2types.CreateFleetInstance{}, cloudprovider.NewCreateError(fmt.Errorf(
"launch templates not found when creating fleet request, %w", err), reason,
fmt.Sprintf("Launch templates not found when creating fleet request: %s",
message))
    }
    return ec2types.CreateFleetInstance{}, cloudprovider.NewCreateError(fmt.Errorf(
"creating fleet request, %w", err), reason, fmt.Sprintf("Error creating fleet
request: %s", message))
}
p.updateUnavailableOfferingsCache(ctx, createFleetOutput.Errors, capacityType,
nodeClaim, instanceTypes)
if len(createFleetOutput.Instances) == 0 || len(createFleetOutput.Instances[0].
InstanceIds) == 0 {
    requestID, _ := awsmiddleware.GetRequestIDMetadata(createFleetOutput.
ResultMetadata)
    return ec2types.CreateFleetInstance{}, errors.Wrap(
        combineFleetErrors(createFleetOutput.Errors),
        middleware.AWSRequestIDLogKey, requestID,
        middleware.AWSOperationNameLogKey, "CreateFleet",
        middleware.AWSServiceNameLogKey, "EC2",
        middleware.AWSStatusCodeLogKey, 200,
        middleware.AWSErrorCodeLogKey, "UnfulfillableCapacity",
    )
}
return createFleetOutput.Instances[0], nil
```

Karpenter Event

真实案例2

场景： 用户想知道为什么xxx node会被删除

传统： 通过prometheus可以定位到是cpu, memory request可能过低, 但很难从此判定, 为什么会有10个node被删除, 又创建了5个node, 只能是猜测

Karpenter event: 可以通过UI或者存储查看controller event, 确认type (disruption/provisioner), node被回收时的resource request, 受影响的业务pod

收益： 正确认识karpenter每一次的操作, 看见karpenter的行为

Karpenter Event

图例

demo-cluster-7c3bc8f6

Jul 01, 2025 08:00 - Aug 01, 2025 08:00

All Operations

Date	Operation	Reason
2025-07-16 14:30:16	delete	node is Underutilized, nodes:...

NodeClaim Events

Date	Type	NodeClaim
2025-07-16 14:30:34	delete	cloudpilot-general-8wgbz

2025-07-16 14:27:33	create	pending pods: 1, pending rea...
2025-07-16 14:02:17	delete	node is Empty, nodes: ip-10-...
2025-07-16 14:00:13	create	pending pods: 1, pending rea...
2025-07-08 23:53:06	replace	node is cloudpilot-disruption...

Showing 1 to 5 of 5 entries Show 10 per page

Event Details

Status	Success
Date	2025-07-16
Cluster Id	4bbab375-e92f-51c6-849c-be0aa959f5df
Provider	aws
Operation	delete
Reason	node is Underutilized, nodes: ip-10-0-36-146.us-east-2.compute.internal
Running Time	18.639197ms

Raw Data

```
{
  "CreatedAt": "2025-07-16T06:30:16.879Z"
  "UpdatedAt": "2025-07-16T06:30:16.879Z"
  "DeletedAt": {2 items}
  "id": "3389698680153932933"
  "userID": "689aa0f423d9170d656a3c2b"
  "clusterID": "4bbab375-e92f-51c6-849c-be0aa959f5df"
  "type": "disruption"
  "operationType": "delete"
  "provider": "aws"
  "reason": "node is Underutilized, nodes: ip-10-0-36-146.us-east-2.compute.internal"
  "nodeClaimNames": [1 items]
  "nodeClaimIDs": [1 items]
```

NodeClaim Details

```
{
  "clusterID": "4bbab375-e92f-51c6-849c-be0aa959f5df"
  "type": "delete"
  "nodeClaimName": "cloudpilot-general-8wgbz"
  "nodeName": "ip-10-0-36-146.us-east-2.compute.internal"
  "provider": "aws"
  "startTimestamp": 1752647433947943200
  "endTimestamp": 1752647434403407400
  "processTime": "455.464051ms"
  "owner": ""
  "status": true
  "errs": null
  "data": {
    "providerInfo": {
      "provider": {"DryRun": null, "InstanceIds": ["i-0e5e364f50ff81cf1"]}
      "provisioning": {"TerminatingInstances": [{"CurrentState": "shutting-down", "InstanceID": "i-0e5e364f50ff81cf1", "PreviousState": "running"}]}
    }
    "nodeClass": null
    "nodeClaim": {
      "apiVersion": "karpenter.sh/v1"
      "kind": "NodeClaim"
      "metadata": {12 items}
      "spec": {4 items}
      "status": {7 items}
    }
  }
}
```

Karpenter Event

隐私、安全与成本考虑

- 数据清洗避免泄露敏感字段
- 异步上报避免影响集群稳定性, 完全不影响主流程
- 计算成本: 使用patch 代码的方式, 构建于karpenter中, 且额外计算压力极大
- 网络传输成本: 已使用gzip压缩, 较大的单个controller event 大小大概在2kb左右

QA

欢迎对此方案有兴趣, 或者有疑问的朋友提问

